WALL-E and EVE's Signal

What I built, how it works, and how to explain it

Prepared for: Vutukuru Saketh Ram (26BCE0804), BTech CSE

Project 2: autonomous agent that explores a simulated environment, gathers clues and manages limited resources to locate EVE

Deliverable (from the brief)	Where it is
Code repo	wall_e_eve/ (Python package **walle/**, plus run.py, benchmark.py, tests/)
README: environment design, config format, run instructions	README.md
Sample configs with traces and reports	configs/ (3 missions) and outputs/<mission>/ (trace.jsonl, report.md, report.json)
Visualization output	outputs/<mission>/: **map_3d.html (interactive 3D map)**, map_3d.png, mission_overview.png, animation.gif, ascii_final.txt
Web control panel	Press Run on run.py (or python run.py): opens a page with a button per mission, 3D map, trace and report
Bonus: noisy sensors, decoys, generalization benchmark	Done (see section 8). 3D/Gazebo, multi-agent and LLM comparison are NOT done.

The one-paragraph version. The agent keeps a probability map of where EVE might be. Every scan is a noisy measurement that updates that map with Bayes' rule. To decide the next move, it scores every reachable cell by how much a scan there would teach it per unit of battery, and it never goes anywhere it cannot return from. When the battery only covers the way home it turns back and recharges. Everything it does is logged with a reasoning sentence, and at the end it writes a report with the evidence, the inferred location, a confidence, and a recommended next step.

1. The problem in plain words

You have a robot with a small battery, a vague radio hint ("EVE is around (12, 8), strength 0.7"), and a map. EVE is hidden. Scanning costs battery, walking costs battery, and a recharge only works at the charger. So you cannot just wander: every step must be worth its cost, and the robot must keep enough battery to get back.

That makes it a **search under uncertainty with a budget**. The standard clean way to solve it is: (a) represent what you believe as probabilities, (b) update them with each observation, (c) pick actions that reduce uncertainty fastest per unit cost.

Brief requirement	Module	How it is met
1. Environment loads config, returns observations	walle/env.py, walle/config.py	Environment class reads JSON/YAML; step(action) returns battery, time, position and scan results. Truth is hidden.
2. Agent chooses move / scan / recharge / turn back	walle/agent.py	Utility = information gained per battery cost, restricted to cells it can still return from.
3. Belief updates, plans adapt to evidence	walle/belief.py	Probability grid updated by Bayes rule after every scan and clue; agent replans after every scan.
4. Full action trace	walle/runner.py	trace.jsonl: action, observation, decision tag, reasoning text and belief summary for every step.
5. Final mission report	walle/report.py	Evidence, inferred location/status, confidence, recommended next step (report.md and report.json).

2. Project layout

File	What it does
walle/config.py	Defaults, JSON/YAML loading, validation, signal-to-coordinates conversion
walle/env.py	The simulated world: grid, obstacles, hidden EVE and clues, costs, scan physics, BFS pathfinding
walle/belief.py	The probability grid and its Bayesian update rules (signal, clues, not-detected)
walle/agent.py	The decision-maker: information-gain planner, battery safety, recharge and turn-back logic
walle/report.py	Builds the final report and the recommended next step
walle/viz.py	ASCII map and the 4-panel matplotlib figure
walle/runner.py	Main loop (agent acts, environment answers) and writing all output files
walle/map3d.py	Writes the interactive 3D map (one self-contained HTML file per mission)
walle/anim.py / animate.py	Heartbeat-sensor style animated GIF of a mission
walle/webui.py	The local web control panel (standard library server + one-page UI)
run.py / benchmark.py	run.py with no arguments opens the web panel; with a config it runs from the command line. benchmark.py runs many random missions
configs/	mission_01.yaml (easy), mission_02 (walls, noise, spoofs, decoys), mission_03.json (bearing + range)
tests/test_core.py	Checks belief maths, hidden truth, cost accounting, and that the agent never strands itself

The design rule that matters most: **the agent only sees what step() returns**. EVE's position and the clue layout live inside the Environment object and are never passed to the agent, so the "learn clue locations only through interaction" requirement is enforced by the code structure, not by trust. A test (test_env_hides_truth_and_charges_costs) checks this.

3. Environment design

World. A 20x20 grid (configurable), 4-connected moves, walls from the config (an explicit list or a random density). The start cell is also the charger. EVE and the clues are reachable cells; if you do not give EVE a position, it is sampled from the same distribution the initial signal implies, so the hint is statistically honest.

Action	Battery	Time	Notes
move	1	1	Adjacent free cell only
scan	3	2	Returns a signal reading, reveals nearby clues, confirms EVE if close
recharge	0	10	Only at the charger, restores full battery, limited by max_recharges
declare	0	0	Ends the mission so the report can be written

What a scan returns.

Observation	How it is generated
Signal reading	exp(-distance to EVE / length_scale) + Gaussian noise. Close to EVE the reading is near 1, far away near 0.
Spoofed reading	With probability spoof_prob the reading is replaced by a random number (a decoy signal).
Clues	Any not-yet-found clue within scan_radius is revealed. A clue gives an estimated EVE position. Reliable clues are near EVE's true position; decoys point to a random cell. The agent is NOT told which is which.

Observation	How it is generated
Detection	If EVE is within detect_radius the scan confirms EVE's position and status.

4. Config format

Configs are JSON or YAML. Everything has a default, so a minimal config is just a map and a signal. Example (mission_01.yaml):

```
name: mission_01_open_field
seed: 7
map: {width: 20, height: 20, obstacles: []}
start: [0, 0]
signal: {coords: [12, 8], strength: 0.7, timestamp: 0}
budget: {battery: 100, time: 300, max_recharges: 3}
costs: {move: 1, scan: 3, move_time: 1, scan_time: 2, recharge_time: 10}
sensor: {length_scale: 6.0, noise: 0.08, spoof_prob: 0.0, scan_radius: 3.0, detect_radius: 1.5}
clues: {count: 5, decoy_prob: 0.2, sigma: 1.5}
```

Key	Meaning	Why it matters
seed	Reproducibility	Same seed = same world, same noise: traces can be re-created
signal.coords or bearing_deg + range	Initial hint	Covers the brief's "direction/coordinates". Bearing 0 = east, 90 = south
signal.strength, timestamp	Hint quality	Weak or old hints widen the prior (prior sigma = (2 + 8 x (1 - strength)) x (1 + 0.03 x age) cells)
budget	battery, time, max_recharges	The resource limits the agent must respect
costs	Cost of each action	Changing these changes the best strategy
sensor	Noise, spoof probability, radii	Makes the sensor trustworthy or messy
clues	count, decoy_prob, or explicit list	Controls how many clues exist and how many lie
agent	policy, safety_margin, stop_confidence, decoy_prior	Agent tuning; policy eig or greedy for comparison

5. The agent's brain

5.1 Belief: where is EVE probably?

The belief is a 20x20 grid of probabilities that sum to 1. At the start it is a bell curve centred on the signal coordinates, plus 10% spread uniformly so one wrong hint cannot lock the agent out of the rest of the map. After that, three kinds of evidence update it:

Evidence	Update (Bayes: new belief = old belief x likelihood, then normalise)
Scan reading r taken at cell p	For each possible EVE cell c, likelihood = 0.9 x Normal(r; exp(-dist(p,c)/6), noise) + 0.1 x (random value). Cells whose expected reading matches r grow, others shrink. The second term is the spoof guard.
Clue pointing at point q	Likelihood = 75% Gaussian bump around q (clue is reliable) + 25% flat (clue is a decoy). One bad clue can bend the belief only a little, and agreeing evidence outvotes it.
Scanned, EVE not seen	Cells within the detection radius of the scan point are multiplied by 0.001 (EVE is not there).
EVE detected	Belief collapses to the confirmed cell.

5.2 Choosing where to scan: information per battery

For each candidate cell the agent asks: *if I walk there and scan, how much will I learn?* It computes the **expected information gain** (the expected drop in the belief's entropy, i.e. the mutual information between the reading and EVE's location), using the same sensor model the environment uses. It adds a bonus for the

chance of actually spotting EVE (probability mass within the detection radius, times current entropy, because spotting EVE ends all uncertainty). Then it divides by the battery cost of getting there and scanning:

```
utility(cell) = ( info_gain(cell) + P(EVE near cell) x current_entropy ) / ( moves_to_cell x move_cost + scan_c
```

It picks the best cell, plans the shortest path around walls (BFS), walks there, scans, updates the belief, and **replans**. Replanning after every scan is what makes the plan adapt to new evidence: in mission_02 the first target was far from where the belief ended up, because the early readings and clues pulled the belief toward a different part of the map.

5.3 Resource rules: reserve, turn back, recharge

Situation	Rule	Where in code
Choosing a target	Only cells where battery >= trip there + scan + trip home to the charger + safety_margin (and enough time). This guarantees WALL-E can always get home.	_replan(): feasible()
Away from the charger and nothing is affordable	TURN BACK: path to charger, logged as a turn-back decision.	_replan()
At the charger	RECHARGE if a full battery would allow a scan at least 1.3x better than anything affordable now, and recharges and time remain.	_replan(): can_recharge
At the charger, nothing useful, no recharge	DECLARE and report the best estimate.	_replan()
EVE confirmed, or belief converged to 95%	DECLARE.	act()

6. Trace and report

Every action produces one JSON line in trace.jsonl, with these fields:

Field	Content
step, time_left, battery, pos_after	State after the action
action	move (with target cell), scan, recharge or declare
decision	replan (new plan chosen), move, scan, turn_back, recharge, declare
reasoning	A sentence explaining why (candidate cell, info gain, probability, cost, battery reserve)
observation	Scan reading, clues found, detection result
belief	Peak cell, peak probability, entropy, confidence, number of cells holding 90% of the probability

A real example from mission_02 (a replan decision):

```
step 2  decision=replan  battery=76
REPLAN: best scan spot (5, 13) (info gain 0.41 nats, P(EVE within 1.5 cells)=0.02, cost 21 battery).
 Belief peak (15, 14) p=0.005, entropy 5.69 nats; 363 of 374 cells affordable with return-to-charger
 reserve.
```

The final report (report.md) contains: outcome (CONFIRMED / PROBABLE / POSSIBLE / UNKNOWN), inferred location, confidence, the 90% credible region size, EVE's status, resources used, the evidence table (every clue marked consistent or conflicting with the final location, plus readings that look spoofed), a recommended next step, and an "evaluation only" block that compares against ground truth. That last block is for you to check the agent; the agent never sees it.

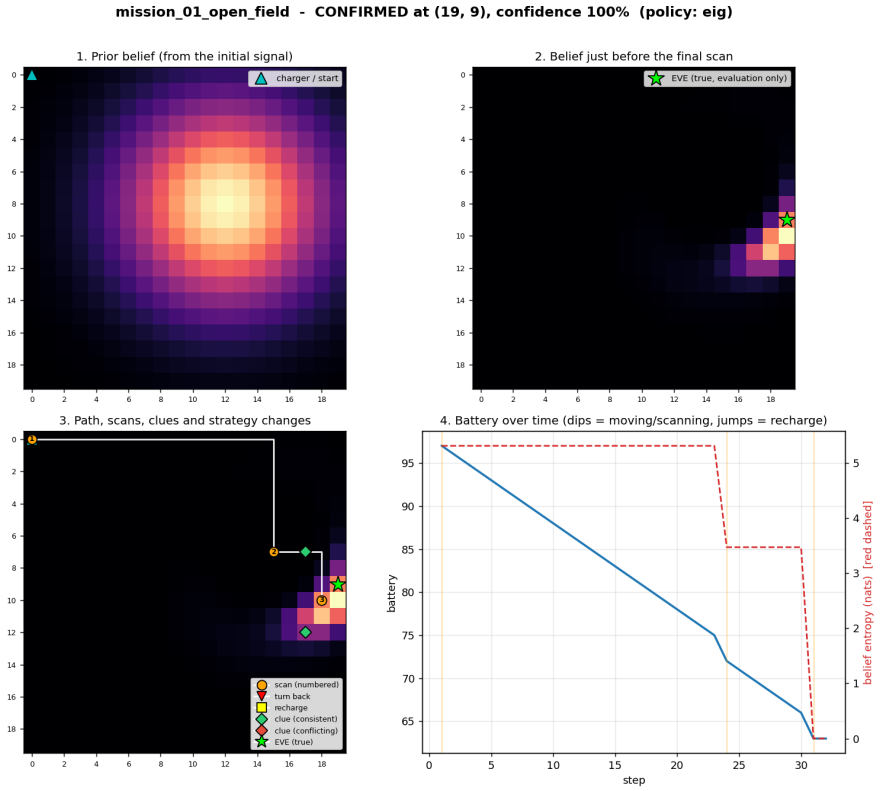
How the recommended next step is chosen: if EVE is confirmed and the battery covers a round trip, go to EVE; if not, return and recharge first; if EVE is only probable, go verify with a close scan; if nothing is conclusive, list the three most likely unexplored cells.

7. Results

Mission	Outcome	Location	Conf.	Steps	Scans	Recharges	Battery left	Error (cells)
01 open field	CONFIRMED	[19, 9]	100%	32	3	0	63/100	0.0
02 walls, noise, decoys	CONFIRMED	[18, 4]	100%	105	6	1	39/80	0.0
03 bearing, JSON	CONFIRMED	[9, 4]	100%	50	6	0	19/80	0.0

7.1 Mission 01 (easy)

The agent scanned at the start (a cheap 3-battery scan), walked toward the belief centre, scanned at (15, 7), saw that the signal was strong to the south-east, moved to (18, 10) and spotted EVE at (19, 9). Total: 32 steps, 3 scans, no recharge needed.



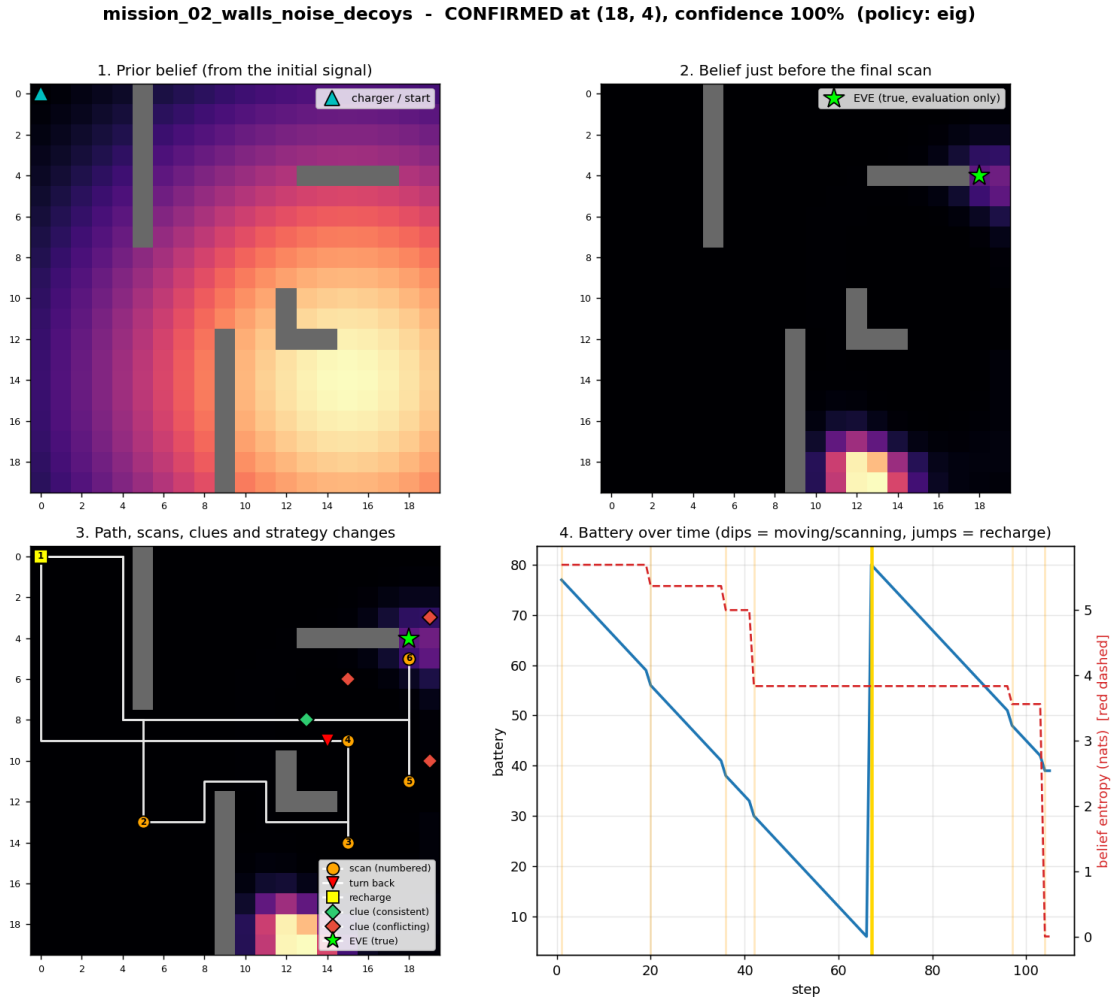

```
   0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9
 0 C * * * * * * * * * * * * * * . . . .
 1 . . . . . . . . . . . . . . * . . . .
 2 . . . . . . . . . . . . . . * . . . .
 3 . . . . . . . . . . . . . . * . . . .
 4 . . . . . . . . . . . . . . * . . . .
 5 . . . . . . . . . . . . . . * . . . .
 6 . . . . . . . . . . . . . . * . . . .
 7 . . . . . . . . . . . . . . S * ? * .
 8 . . . . . . . . . . . . . . . . * .
 9 . . . . . . . . . . . . . . . . * E
10 . . . . . . . . . . . . . . . W .
11 . . . . . . . . . . . . . . . . . .
12 . . . . . . . . . . . . . . ? . . .
13 . . . . . . . . . . . . . . . . . .
14 . . . . . . . . . . . . . . . . . .
15 . . . . . . . . . . . . . . . . . .
16 . . . . . . . . . . . . . . . . . .
17 . . . . . . . . . . . . . . . . . .
18 . . . . . . . . . . . . . . . . . .
19 . . . . . . . . . . . . . . . . . .
```

Final ASCII map of mission 01. C = charger, W = WALL-E, * = path, S = scan, ? = clue found, E = EVE (true position, shown for evaluation).

7.2 Mission 02 (hard: walls, noise, spoofed readings, decoy clues)

Here the battery is 80, the signal is weak and stale (strength 0.4, 12 ticks old), the sensor noise is 0.12, 10% of readings are spoofed, 40% of clues are decoys, and walls make paths long. This is the mission that exercises the resource logic. Key moments from the trace:

Step	Decision	What happened
2	replan	Chose the first far scan target from the weak prior
43	turn back	TURN BACK: battery 30 only covers the return trip (24 moves) plus margin 5.
67	recharge	At the charger with battery 6/80: no useful scan is reachable with the current battery. Recharging (3 recharge(s) left).
105	declare	EVE visually confirmed at (18, 4) (status: active). Mission goal reached.



Panel 4 shows the battery draining, the jump at the recharge, and the belief entropy (red dashed) falling in steps as scans come in. Panel 3 marks scans (orange, numbered), the turn-back (red triangle), and clues (green = consistent with the final location, red = conflicting, likely decoys).

The mission 02 report flagged 3 of 4 found clues as conflicting with the final location. The world actually contained 4 decoy clues in total, so the belief was not dragged off by them.

7.3 Benchmark: does it generalize? (40 random missions)

benchmark.py builds 40 random missions (random signal position and quality, 0-15% obstacles, battery 50/70/100, noise 0.05-0.12, spoof 0-10%, decoy clues 20-40%) and runs both planners on exactly the same missions. The baseline (greedy) just walks to the cell with the most probability mass and scans; it uses the same belief updates and the same battery safety rule, so the difference is only the action-selection rule.

Metric	EIG planner (mine)	Greedy baseline
Missions	40	40
EVE visually confirmed	80.0%	77.5%
Final answer within 1.5 cells of truth	92.5%	90.0%
Mean error (cells)	0.26	0.47
Mean steps	63.7	80.8
Mean scans	4.35	5.62
Mean recharges	0.82	1.07

Honest reading of this table. The EIG planner finds the answer with about 21% fewer steps (63.7 vs 80.8) and a smaller average error, but the success rates differ by only 1 or 2 missions out of 40, which is within noise. So the safe claim is: *it is more efficient and at least as accurate as the baseline*, not that it is dramatically better. The raw per-mission data is in outputs/benchmark/benchmark.csv if you want to examine failures.

8. Bonus items: done vs not done

Bonus item from the brief	Status
Visualize path, clue locations, strategy changes (ASCII, UI, 3D)	Done: interactive 3D map, ASCII map, 4-panel matplotlib figure, heartbeat-sensor animation, web UI
Noisy sensors	Done: Gaussian noise, configurable
Decoy / spoofed signals	Done: spoofed readings and decoy clues, handled with mixture likelihoods; report flags suspected spoofs
Generalization benchmark across random missions	Done: benchmark.py
LLM vs classical planner comparison	Not done. I compared two classical planners (EIG vs greedy) instead
Multi-agent coordination	Not done
Physics simulation (Gazebo/Webots), 3D	Not done

9. Visual outputs: 3D map, heartbeat animation, web control panel

These three were added after the core agent. They do not change how the agent thinks; they make the mission easy to see and to demo. The Round 1 note says the Wall-E path must be visualized with a graph, 3D map or similar (JSON alone is not enough), so the 3D map is the main one for that.

9.1 The 3D map (map_3d.html)

Each mission writes one self-contained file, outputs/<mission>/map_3d.html. Double-click it: it opens in any browser, needs no server and no internet. It is built with plain HTML canvas and my own projection maths (no 3D library), so it also works offline on a lab computer.

What you see	What it means
Rising columns, blue to yellow to white	The agent's belief about where EVE is. Taller and brighter = more likely. Watch the hill shrink onto one spot as scans arrive
Glowing orange blocks	Obstacles (walls)
Blue tile labelled BASE	The charger and start cell
Cyan line with orange dots	WALL-E's trail; the dots are the scan spots
Cyan beacon with a W	WALL-E's current position (always drawn on top so columns never hide it)
Yellow ? markers with dashed lines	Clues found, and the point each clue claims EVE is at. A decoy's line points far away from the hill
Green expanding rings	The scan ping. Brighter ring = stronger signal return
Red rings and a white pillar	EVE confirmed (lock-on)

Control	Action
Play / Pause / Step / Replay, speed 0.5x to 4x, scrubber	Replay the mission frame by frame; each frame shows the agent's reasoning text and battery
Belief Heatmap, Rover Trail, Clues, Ground Truth	Layer toggles. Ground Truth shows EVE and every clue (green = reliable, red = decoy). It is for evaluation: the agent never sees it
Iso, Top, Low angle, Auto-rotate; drag; wheel	Camera controls

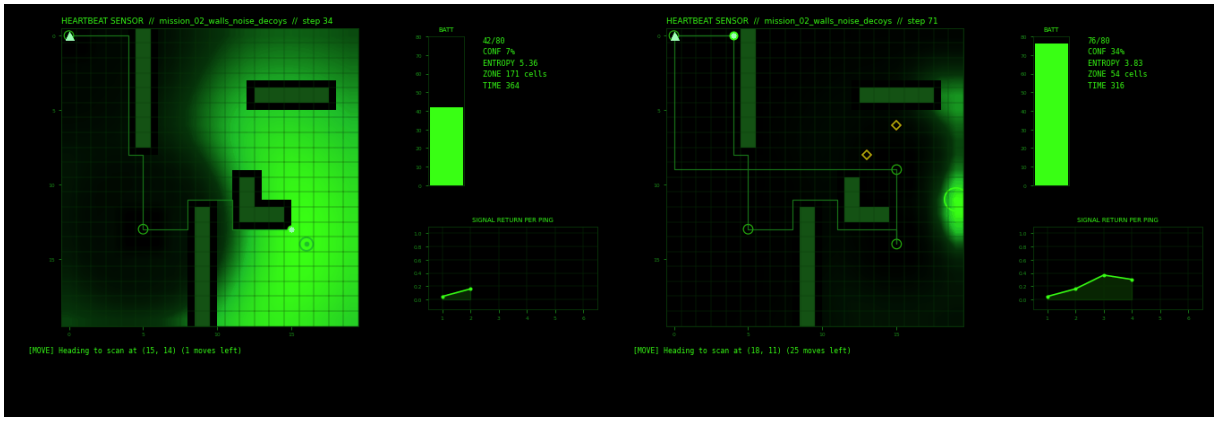
How it is drawn (for the viva). Every grid cell becomes a box; belief value controls its height (height = 0.08 + 3.4 x belief^0.9, where belief is scaled so the largest is 1). Each 3D point is rotated by the yaw and pitch angles, then divided by depth for perspective. Boxes are sorted far to near (painter's algorithm) and only faces pointing toward the camera are drawn. The belief shown at each frame is the agent's real belief at that step, saved by the runner.



Mission 02 at the last frame: trail with the turn-back and recharge detour, three clue pointers disagreeing with the others (decoys), and the lock on EVE at (18, 4).

9.2 Heartbeat-sensor animation (animation.gif)

Same mission as a green-on-black radar screen, like a heartbeat sensor. Every scan sends out expanding rings from WALL-E; brighter rings mean a stronger return. The belief glows green, a pulsing lub-dub blip sits on the current best guess and grows with confidence, and a small strip chart records the signal return of each ping. Make one with: python animate.py configs/mission_02_walls_noise_decoys.yaml



Two frames from mission 02: early (confidence 7%, large uncertain zone) and later (confidence 34%, belief concentrated on the right edge, ping ring visible).

9.3 The web control panel

Press Run on run.py (or type python run.py with no arguments). A small web server starts on your own computer (127.0.0.1, nobody else can reach it) and your browser opens the control panel. Everything it runs is the same code as the command line, so results are identical.

Control	What it does
RUN on a mission card	Runs that mission; shows the result badge, stats, recommended next step and tabs: 3D Map, Overview, Animation, Step trace, ASCII map, Report
Planner switch (EIG / Greedy)	Choose my information-gain planner or the greedy baseline for the same mission
Seed box and RND	New seed = EVE and the clues are hidden somewhere else, so you get a fresh test case from the same config
Also make heartbeat animation	Builds the GIF too (10 to 30 seconds)
Run all missions	Runs every config and shows one comparison table
Compare planners (benchmark)	Runs 10, 20 or 40 random missions for both planners
Run unit tests	Shows PASS or FAIL for each test
Quit server	Stops the server (Ctrl+C in the terminal also works)



The panel running mission 02 with the 3D map tab open. Under the hood it uses only Python's standard library: the page calls small endpoints (/api/run, /api/animate, /api/benchmark, /api/tests), jobs run in a background thread one at a time, and the page polls for the result.

10. How to run it

```
pip install -r requirements.txt            # numpy, matplotlib, pyyaml
python run.py                                  # opens the web control panel (easiest)
python run.py configs/mission_02_walls_noise_decoys.yaml -v   # command line
python run.py configs/mission_01.yaml --policy greedy         # baseline planner
python animate.py configs/mission_01.yaml        # heartbeat animation
python benchmark.py --n 40                       # planner benchmark
PYTHONPATH=. python tests/test_core.py          # tests (Mac/Linux)
$env:PYTHONPATH="."; python tests/test_core.py   # tests (Windows PowerShell)
```

Open outputs/<mission>/map_3d.html by double-click for the 3D map.

Outputs land in outputs/<config name>/. To make your own mission, copy a config and change the signal, budget or map.

11. Limitations you should know about (and can mention)

Limitation	Why it is that way
The first action in mission 01 is a scan at the charger, where the reading is almost zero	It is cheap (3 battery) and still tells the agent something (EVE is far). It is a consequence of dividing information by cost; you could add a minimum-information threshold if you dislike it.
The planner is myopic (one scan ahead)	Full multi-step planning is much more expensive. Replanning after every scan recovers most of the benefit.
EVE does not move	The brief says nothing about a moving target. The timestamp only widens the prior.
Signal model is a simple exponential with no wall blocking	Kept simple so the likelihood is easy to explain. The same belief machinery would work with a better model.
Benchmark has 40 missions	Enough to see a trend, not enough to prove a small difference in success rate. Raise --n for tighter numbers.
Mission 02 can end with a lower confidence if it is run with a smaller battery	With battery 55 the same mission ends POSSIBLE at 57%: each trip yields one scan. That is the resource limit working as designed.

12. If your teacher or judge asks...

Question	Short answer
How does the agent learn clue locations only through interaction?	Clues live inside the Environment. Scans reveal clues within a radius; the agent never receives the list.
How does it handle fake clues or spoofed signals?	Every update is a mixture: probability p the evidence is honest, 1-p it is random. Agreeing evidence outweighs a liar, and the report flags conflicting items.
Why not just move to the highest-probability cell?	That is the greedy baseline. It ignores cost and ignores that a scan from a different spot can be more informative. The benchmark shows it needs about 27% more steps.
How does it decide when to turn back?	Every target must leave battery for the return trip plus a margin. If no target satisfies that, it heads home.
How does it decide to recharge?	At the charger, if a full battery unlocks a scan at least 1.3x as valuable as anything affordable now.
What is your confidence number?	The belief probability inside the detection radius of the best cell (1.0 once EVE is actually seen).
Does the 3D map leak where EVE is? Is it a library?	No leak: the columns are the agent's own belief; EVE appears only when confirmed or with the evaluation-only Ground Truth layer. No library: HTML canvas plus my own rotation and perspective maths, one offline file.
What would you add next?	Moving EVE (particle filter), line-of-sight blocking in the signal, multi-step lookahead, and an LLM planner for the comparison bonus.